Security Assessment Report

AAVE V3.7

March 2026

Code developed by:

BORED
GHOSTS
DEVELOPING

Table of Contents

Project Summary **3**
Project Scope 3
Project Overview 3
Protocol Overview 3
Assessment Methodology **4**
Threat and Security Overview **5**
Findings Summary 7
Severity Matrix 7
Detailed Findings **8**
Informational Severity Issues 9
I-01: Emergency admin can disable eMode isolation and restore borrowing power 9
I-02: If LTV0 is not already configured, users may take risky loans against previously isolated assets 11
Disclaimer **12**
About Certora **12**

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
AAVE V3.7	https://github.com/aave-dao/aave-v3-origin	PR#38 75145ad (last commit)	Solidity

Project Overview

This document describes the security review of **AAVE V3.7**. The work was undertaken from **March 10th, 2026** to **March 13th, 2026**.

The following contract list is included in our scope:

```
src/contracts/interfaces/*
src/contracts/protocol/*
```

The team performed a manual audit of all the files in scope**.** During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

This PR simplifies the protocol by removing several unused and deprecated components, and refining existing mechanisms. It removes the legacy **Isolation Mode** and **Siloed Borrowing** logic, eliminates the **L2 sequencer sentinel dependency** from borrow and liquidation validations, and removes dropReserve() to make the reserve list append-only and avoid reserve index reuse.The primary change introduces an **isolated eMode** configuration, adding an isolated flag to eMode categories that restricts borrowing power to assets not listed in the category's collateralBitmap. This simplifies LTV0 configuration of some eModes, and provides a simpler mechanism to replace the legacy isolation mode. The PR also improves liquidation rounding logic to ensure deterministic rounding and remove potential rounding bias.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

Upgrade-related risks examined:

Improper removal of features:

- Dangling references left.
- Dead code left.
- Required protections were not replaced.
- Backwards storage compatibility was not kept.
- Relevant removed features:
  - isolation mode
  - siloed borrowing mode
  - drop reserve logic
  - PriceOracleSentinel
- Dangerous collateral-borrow pairs may be used, which were previously prevented by isolation mode and siloed borrowing.

Isolated eMode:

- Regular eMode risks:
  - borrowing outside eMode.
  - HF when entering/exiting eMode
- Users may supply assets outside an isolated eMode as collateral.
- eMode isolation configuration is open to non-privileged actors.

Liquidation changes:

- Bad accounting of overall funds.

- Rounding against the liquidator may make liquidation unprofitable.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	-	-	-
Low	-	-	-
Informational	2	2	-
Total	2	2	-

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

Detailed Findings

ID	Title	Severity	Status
I-01	Emergency admin can disable eMode isolation and restore borrowing power	Informational	Acknowledged
I-02	If LTV0 is not already configured, users may take risky loans against previously isolated assets	Informational	Acknowledged

Informational Severity Issues

I-01: Emergency admin can disable eMode isolation and restore borrowing power

Files:

`protocol/pool/PoolConfigurator.sol`

Description:

setEModeCategoryIsolated() is protected by onlyRiskOrPoolOrEmergencyAdmins, which allows an emergency admin to both enable and disable eMode isolation.

```
function setEModeCategoryIsolated(
  uint8 categoryId,
  bool isolated
) external onlyRiskOrPoolOrEmergencyAdmins {
  _pool.configureEModeCategoryIsolated(categoryId, isolated);
  emit EModeCategoryIsolationChanged(categoryId, isolated);
}
```

These two actions have materially different risk profiles:

- `isolated = true` is restrictive. Assets outside the eMode collateral bitmap become LTV-zero, reducing borrowing power.
- `isolated = false` is expansive. Assets outside the bitmap immediately recover their base LTV, which can increase borrowing capacity system-wide for users already inside that eMode.

Allowing emergency admins to perform the disable operation as well grants them the ability to execute a protocol-wide risk-expanding change. If the emergency admin role is intended to be limited to defensive or risk-mitigating actions, permitting this capability may exceed the intended scope of that role, as disabling isolation restores borrowing power across existing positions within the affected eMode category.

Recommendations:

Restrict the privilege asymmetrically:

allow emergency admins to set isolated = true,

restrict isolated = false to risk admin or pool admin only

```
function setEModeCategoryIsolated(uint8 categoryId, bool isolated) external {
    if (isolated) {
        _onlyRiskOrPoolOrEmergencyAdmins();
    } else {
        _onlyRiskOrPoolAdmins();
    }

    _pool.configureEModeCategoryIsolated(categoryId, isolated);
    emit EModeCategoryIsolationChanged(categoryId, isolated);
}
```

Customer Response:

This is by design.

I-02: If LTV0 is not already configured, users may take risky loans against previously isolated assets

Files:

`src/contracts/protocol/libraries/logic/BorrowLogic.sol`
`docs/3.7/mode-removal.md`

Description:

Per the documentation (Effects, Minimal Impact, Isolation Mode (Debt Ceiling)), all isolated modes should already be LTV0. If however any asset was missed in this check, users holding it will be able to borrow any asset against it after the upgrade (as the upgrade process does not include any enforcement that previously isolated assets are LTV0).

Recommendations:

It would be prudent to enforce in the upgrade AIP that any currently isolated asset will be LTV0 after the upgrade, to avoid potential dangerous borrowing in case of a current misconfiguration. This should also be documented in the "Upgrade Process" section of mode-removal.md.

Customer Response:

We will enforce this on the upgrade proposal.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.